

# CS 621

## Scalable AI Services

### QUIZ 1 · COMPLETE PREPARATION GUIDE

PySpark · Delta Lake · Spark Catalog · Unity Catalog · Delta Sharing

LUMS · Syed Babar Ali School of Science and Engineering

| Chapter | Topic                           | Key Concepts                                     |
|---------|---------------------------------|--------------------------------------------------|
| CH 1    | PySpark DataFrames & Operations | Filters, Joins, Aggregations, MERGE, Window Fns  |
| CH 2    | Delta Lake — Core Concepts      | ArrayType, StructType, Schema, CoW vs MoR        |
| CH 3    | Delta Lake — Advanced Features  | VACUUM, Z-Ordering, Time Travel, Partitioning    |
| CH 4    | Spark Architecture & Storage    | In-Memory Processing, Row vs Columnar            |
| CH 5    | Spark Catalog & Hive Metastore  | Components, Issues, Setup, Managed vs External   |
| CH 6    | Databricks Unity Catalog        | 3-Level Namespace, Lineage, Volumes, Governance  |
| CH 7    | Delta Sharing                   | Open Protocol, D2D, Open Sharing, Under the Hood |

# CHAPTER 1

*PySpark DataFrames & Operations*

## 1.1 Quiz DataFrames — Know These COLD

**products\_df:**

| product_id | product_name | category    | price |
|------------|--------------|-------------|-------|
| 101        | Laptop       | Electronics | 1200  |
| 102        | Smartphone   | Electronics | 800   |
| 103        | Desk Chair   | Furniture   | 150   |
| 104        | Monitor      | Electronics | 300   |
| 105        | Backpack     | Travel      | 50    |

**sales\_df:**

| sale_id | product_id | quantity | sale_date  |
|---------|------------|----------|------------|
| 1       | 101        | 2        | 2024-01-01 |
| 2       | 102        | 1        | 2024-01-02 |
| 3       | 101        | 1        | 2024-01-03 |
| 4       | 103        | 4        | 2024-01-04 |
| 5       | 105        | 2        | 2024-01-05 |
| 6       | 999 ■      | 1        | 2024-01-06 |

### ■ EXAM TRAP

product\_id 999 has NO match in products\_df — this is the trick row in EVERY join question!

## Q1 — filter + select + orderBy (Expected Output)

Filter products with price  $\geq 300$ , select name and category, order by price DESC.

```
df.filter(col("price") >= 300)
  .select("product_name", "category")
  .orderBy(col("price").desc())
```

# OUTPUT:

```
| product_name | category |
```

```
| Laptop | Electronics | ← 1200
| Smartphone | Electronics | ← 800
| Monitor | Electronics | ← 300
```

#### ■ KEY CONCEPT

Laptop=1200, Smartphone=800, Monitor=300 all pass  $\geq 300$ . Backpack(50) and Desk Chair(150) are excluded. Default orderBy is ASC — always use .desc() explicitly.

## Q2 — Inner Join + groupBy + agg (Expected Output)

Inner join sales\_df with products\_df, group by category, sum quantity.

```
# Inner join DROPS sale_id=6 (product_id 999 has no match)
joined = sales_df.join(products_df, "product_id", "inner")
result = joined.groupBy("category").agg(sum("quantity").alias("total_items"))
.orderBy("total_items")

# OUTPUT:
| category | total_items |
| Travel | 2 | ← sale 5 (qty 2)
| Furniture | 4 | ← sale 4 (qty 4)
| Electronics | 4 | ← sales 1+2+3 (2+1+1)
```

#### ■ KEY CONCEPT

Electronics =  $2+1+1 = 4$ . sale\_id 6 (product 999) is DROPPED by inner join. orderBy total\_items ASC is the default.

## Q3 — withColumn + when/otherwise + filter (Expected Output)

Add discounted\_price: Electronics 10% off, filter out Travel.

```
from pyspark.sql.functions import when, col
df.withColumn("discounted_price",
when(col("category") == "Electronics", col("price") * 0.9)
.otherwise(col("price")))
.filter(col("category") != "Travel")

# OUTPUT:
| product_name | discounted_price |
| Laptop | 1080.0 | ←  $1200 \times 0.9$ 
| Smartphone | 720.0 | ←  $800 \times 0.9$ 
| Desk Chair | 150.0 | ← unchanged
```

```
| Monitor | 270.0 | ← 300×0.9
```

```
# Backpack/Travel is filtered OUT
```

## Q4 — Delta MERGE (UPSERT) ★ 10 Marks ★

updates\_df: product\_id 102 → price 850, product\_id 106 → price 200 (new).

```
from delta.tables import DeltaTable

deltaTable = DeltaTable.forPath(spark, "/path/products")

deltaTable.alias("t").merge(
    updates_df.alias("s"),
    "t.product_id = s.product_id"
).whenMatchedUpdate(set={"price": "s.price"})
.whenNotMatchedInsert(values={
    "product_id": "s.product_id",
    "product_name": "New Item",
    "category": "Misc",
    "price": "s.price"
}).execute()

# AFTER MERGE:

| 101 | Laptop | Electronics | 1200 | ← unchanged
| 102 | Smartphone | Electronics | 850 | ← UPDATED
| 103 | Desk Chair | Furniture | 150 | ← unchanged
| 104 | Monitor | Electronics | 300 | ← unchanged
| 105 | Backpack | Travel | 50 | ← unchanged
| 106 | New Item | Misc | 200 | ← INSERTED
```

### ■ KEY CONCEPT

WHEN MATCHED → UPDATE existing row. WHEN NOT MATCHED → INSERT new row. Only product\_id 102 matches (updated). 106 is new (inserted).

## Q5 — Full Outer Join + isNull filter (Expected Output)

```
full_join = products_df.join(sales_df, "product_id", "full")

result = full_join.filter(
    col("product_name").isNull() | col("sale_id").isNull()
).select("product_id", "product_name", "sale_id")
.orderBy("product_id")
```

```
# OUTPUT:
```

```
| product_id | product_name | sale_id |  
| 104 | Monitor | null | ← in products, never sold  
| 999 | null | 6 | ← in sales, no product
```

#### ■ KEY CONCEPT

Full outer keeps ALL rows from BOTH sides. Monitor (104) has no sales → sale\_id is NULL. product 999 has no product → product\_name is NULL.

## Q6 — Left Anti Join (Expected Output)

```
products_df.join(sales_df, "product_id", "left_anti")
```

```
# OUTPUT:
```

```
| product_id | product_name |  
| 104 | Monitor |
```

```
# Only Monitor was NEVER sold
```

```
# IDs 101,102,103,105 all appear in sales → excluded
```

## 1.2 Join Types — Quick Reference

| Join Type  | What It Returns                        | NULL Rows?       | Our Example                |
|------------|----------------------------------------|------------------|----------------------------|
| inner      | Rows matching in BOTH                  | None             | Drops sale 6 (999)         |
| left       | All left + matching right              | Right side NULLs | Keeps all products         |
| right      | All right + matching left              | Left side NULLs  | Keeps all sales            |
| full/outer | All rows from both                     | Both sides       | Shows 104 & 999            |
| left_anti  | Left with NO match in right            | None (excluded)  | Returns Monitor only       |
| left_semi  | Left that HAVE a match (no right cols) | None             | Products that were sold    |
| cross      | Cartesian product                      | None             | Every product × every sale |

# CHAPTER 2

## Delta Lake — Core Concepts

### 2.1 What is Delta Lake?

Delta Lake is an **open-source storage layer** built on Apache Parquet that adds ACID transactions, schema enforcement, and time travel to data lakes.

#### ■ KEY CONCEPT

- Sits between Spark compute and cloud storage (S3 / ADLS / GCS)
- Every Delta table = Parquet data files + a **\_delta\_log** transaction log folder
- **\_delta\_log** stores JSON files (each commit) + periodic Parquet checkpoint files
- **VACUUM** only deletes Parquet files — the **\_delta\_log** is never deleted by **VACUUM**

### 2.2 ArrayType vs StructType (Q1 — Short Answer)

| Feature        | ArrayType                           | StructType                                         |
|----------------|-------------------------------------|----------------------------------------------------|
| Purpose        | List/collection of SAME type values | Nested record with NAMED fields of DIFFERENT types |
| Python Analogy | Like a Python list: [1, 2, 3]       | Like a Python dict: {name: "Ali", age: 22}         |
| Access         | By index: col("phones")[0]          | By field name: col("address.city")                 |
| Use Case       | Tags, phone numbers, list of items  | Nested address, embedded order details             |
| Nesting        | Can contain StructType elements     | Can contain ArrayType fields                       |

#### ■ KEY CONCEPT

StructType is how Spark defines a DataFrame's schema — it is a list of StructField objects. StructType = schema definition!

### 2.3 Schema Enforcement vs Schema Evolution (Q2 — Short Answer)

| Aspect        | Schema Enforcement (default)                    | Schema Evolution (opt-in)                   |
|---------------|-------------------------------------------------|---------------------------------------------|
| Behavior      | Rejects writes that don't match existing schema | Allows new columns to be merged into schema |
| Purpose       | Guard / protect the table                       | Grow / extend the table                     |
| How to enable | Always ON by default — no config needed         | .option("mergeSchema", "true") on write     |

|                         |                                  |                                         |
|-------------------------|----------------------------------|-----------------------------------------|
| Effect on existing rows | Error thrown — write is rejected | New columns appear as NULL for old rows |
|-------------------------|----------------------------------|-----------------------------------------|

```
# Enable Schema Evolution per-write:

df.write.format("delta") \

.option("mergeSchema", "true") \

.mode("append") \

.save("/path/to/delta")

# Enable globally:

spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
```

#### ■ EXAM TRAP

Enforcement = REJECT bad writes (guards). Evolution = ACCEPT new columns (grows). This is a classic short-answer distinction!

## 2.4 Copy-on-Write vs Merge-on-Read (Q3 — Short Answer)

| Aspect            | Copy-on-Write (CoW) — DEFAULT                        | Merge-on-Read (MoR)                                          |
|-------------------|------------------------------------------------------|--------------------------------------------------------------|
| Write Behavior    | Rewrites entire Parquet files on every update/delete | Writes small delta files WITHOUT touching original files     |
| Read Behavior     | Fast — reads clean Parquet files directly            | Slightly slower — merges original + delta files at read time |
| File System After | New clean Parquet files replacing old ones           | BOTH original Parquet files AND small delta files exist      |
| Best For          | Read-heavy workloads                                 | Write-heavy / streaming workloads                            |

#### ■ EXAM TRAP

Why does file system have BOTH files after MoR? Because original data files are IMMUTABLE — they are never rewritten. Delta writes change records separately and merges at READ time only.

# CHAPTER 3

Delta Lake — Advanced Features

## 3.1 VACUUM Command (Q6 — Short Answer)

```
VACUUM products_table RETAIN 168 HOURS; -- 168 hours = 7 days
```

| Days Since Deletion | With RETAIN 168 HOURS (7 days)                   | Can Time Travel? |
|---------------------|--------------------------------------------------|------------------|
| < 7 days            | Within retention window → Files KEPT             | ✓ YES            |
| = 7 days            | At boundary → Files KEPT                         | ✓ YES            |
| 10 days             | Beyond retention → Files DELETED permanently     | ✗ NO             |
| 30 days             | Far beyond retention → Files DELETED permanently | ✗ NO             |

### ■ EXAM TRAP

VACUUM only deletes Parquet data files. The `_delta_log` JSON/checkpoint files are NEVER deleted by VACUUM. Files deleted 10 days ago with 7-day retention = GONE FOREVER.

## 3.2 Z-Ordering (Q7 — 10 Marks)

Z-Ordering is a **multi-dimensional data clustering technique** that co-locates related rows in the same Parquet files to improve query performance through data skipping.

```
OPTIMIZE iot_telemetry ZORDER BY (device_id, city);
```

### Part (a) — Does 2nd query benefit from Z-Order on (device\_id, city)?

Query: `SELECT * FROM iot_telemetry WHERE sensor_type = "Temp" AND city = "Chicago"`

### ■ KEY CONCEPT

- **YES, PARTIAL benefit** — city IS in the Z-Order key → Delta can skip files where `min_city > "Chicago"` or `max_city < "Chicago"`
- `sensor_type` is NOT in the Z-Order → no skipping for that filter
- Full benefit requires ALL filter columns in the Z-Order key

### Part (b) — Z-Ordering vs Standard Partitioning

| Aspect         | Standard Partitioning                                               | Z-Ordering                                                       |
|----------------|---------------------------------------------------------------------|------------------------------------------------------------------|
| File Structure | Separate directories per partition value<br>(/date=2024-01-01/)     | All data in same directory; files internally sorted with Z-curve |
| Cardinality    | Low-cardinality only (date, region) — high cardinality = tiny files | Handles HIGH-cardinality (device_id, user_id) well               |

|              |                                        |                                                      |
|--------------|----------------------------------------|------------------------------------------------------|
| Multi-column | Poor — exponential directory explosion | Excellent — naturally handles multi-column filtering |
| Best For     | Date-range scans, coarse filtering     | Fine-grained multi-column point queries              |

### Part (c) — How Min-Max Statistics + Z-Order Enable Data Skipping

#### ■ KEY CONCEPT

**Step 1:** Z-Ordering clusters data → rows with similar (device\_id, city) written to SAME files → tight min-max range per file

**Step 2:** Statistics stored in Delta Log → min(city), max(city), min(device\_id), max(device\_id) per file

**Step 3:** Query planning → WHERE city="Chicago" → Spark reads log, checks each file's min\_city and max\_city → if "Chicago" is NOT in range → entire file SKIPPED

**Without Z-Order:** Each file has rows from all cities → min="Atlanta", max="Zurich" → no files skipped → full table scan

**With Z-Order:** Each file has rows from nearby cities → tight ranges → many files skipped

# CHAPTER 4

## Spark Architecture & Storage

### 4.1 Spark High-Performance Through In-Memory Processing (Q4)

| Component          | What It Does                                                | Why It Matters                                                   |
|--------------------|-------------------------------------------------------------|------------------------------------------------------------------|
| RDD                | Immutable distributed collection stored in RAM across nodes | Data stays in memory across pipeline — no disk I/O between steps |
| DAG Execution      | Builds Directed Acyclic Graph of all operations             | Executes as ONE pipeline without writing to disk between stages  |
| Lazy Evaluation    | Transformations not executed until an Action is called      | Spark optimizes the ENTIRE plan before running anything          |
| Catalyst Optimizer | Analyzes & rewrites query plan for maximum efficiency       | Automatic optimization before execution                          |
| Tungsten Engine    | Binary memory management + code generation                  | Avoids JVM overhead; processes data faster                       |

#### ■ KEY CONCEPT

Spark = fast in-memory compute. Delta = reliable optimized persistent storage. Together they form an end-to-end high-performance system.

### 4.2 Row-Based vs Columnar Storage (Q5 — Short Answer)

| Aspect               | Row-Based (CSV, JSON, RDBMS)                          | Columnar (Parquet, ORC)                                    |
|----------------------|-------------------------------------------------------|------------------------------------------------------------|
| Storage Layout       | One full row stored together: [id, name, age, salary] | Each column stored separately: [all IDs], [all names]...   |
| Read Pattern         | Read ALL columns even if you only need 2              | Read ONLY the columns you query — huge I/O savings         |
| Compression          | Low — mixed data types in a row compress poorly       | High — same data type per column compresses extremely well |
| Write Speed          | Faster writes — append one row                        | Slower writes — must update multiple column files          |
| Analytics            | Slow — reads unnecessary data                         | Fast — skips irrelevant columns entirely                   |
| OLTP (transactional) | Better — frequent single-row reads/writes             | Poor — designed for bulk reads, not single-row updates     |

#### ■ KEY CONCEPT

Analytics query (`SELECT avg(salary) FROM employees`) needs only 1 column out of 50. Columnar layout reads ONLY that column → ~98% I/O reduction. Delta Lake uses Parquet internally → gets all these benefits automatically.

# CHAPTER 5

## Spark Catalog & Hive Metastore

### 5.1 Why Spark Needs a Catalog

By default Spark has **no persistent memory** of your data. Every session you must manually provide path, format, and schema. The Catalog solves this by:

- Mapping table names → file paths (write SQL instead of file paths)
- Storing schema definitions → no schema inference on every read
- Tracking databases, tables, partitions, views, and functions
- Sharing metadata across sessions/clusters (with Hive Metastore)

### 5.2 Spark Catalog Components

| Component                        | Description                                                                                                          |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------|
| SessionCatalog                   | Entry point — manages lifecycle of temp tables (session-scoped) and delegates to ExternalCatalog                     |
| ExternalCatalog (In-Memory)      | Default non-persistent catalog — dies when SparkSession ends. Good for testing only.                                 |
| ExternalCatalog (Hive Metastore) | Production standard — persistent, shared across clusters and sessions                                                |
| Catalog API                      | Programmatic interface: <code>spark.catalog.listTables()</code> , <code>spark.catalog.tableExists("t")</code> , etc. |

### 5.3 Native Spark Catalog — 3 Problems (Classic Exam Topic)

| Problem                | Issue                                                               | Impact                                                  | Fix                                              |
|------------------------|---------------------------------------------------------------------|---------------------------------------------------------|--------------------------------------------------|
| 1. Single Session Lock | Embedded Derby DB allows only ONE SparkSession at a time            | "Metastore is locked" error when second notebook runs   | Use standalone Hive Metastore (MySQL/PostgreSQL) |
| 2. Metadata Drift      | Files deleted/added directly in S3/HDFS — catalog doesn't know      | FileNotFoundException or missing new data               | REFRESH TABLE or MSCK REPAIR TABLE               |
| 3. Small File Trap     | High-cardinality partitioning (user_id with 1M users) = 1M+ entries | Massive query planning latency before data is even read | Use coarse-grained partitioning (date, region)   |

### 5.4 Spark with Hive Metastore Setup

```
# spark-defaults.conf / SparkConf settings:
```

```
# 1. Tell Spark to use Hive external catalog
spark.sql.catalogImplementation = hive

# 2. Warehouse directory (where managed table data is stored)
spark.sql.warehouse.dir = /user/hive/warehouse

# 3. Connect to external Hive Metastore (PostgreSQL)
spark.hadoop.javax.jdo.option.ConnectionURL = \
jdbc:postgresql://postgres-db:5432/metastore

# Then use standard SQL:
spark.sql("SELECT * FROM users")
```

■ **EXAM TRAP**

Managed Tables: DROP TABLE deletes BOTH metadata AND data files. External Tables: DROP TABLE deletes ONLY the metadata pointer — data remains SAFE.

# CHAPTER 6

## Databricks Unity Catalog

### 6.1 Why Unity Catalog? — HMS Limitations

| HMS Limitation                             | Unity Catalog Solution                                               |
|--------------------------------------------|----------------------------------------------------------------------|
| Per-workspace/per-cluster — metadata silos | Single metastore shared across ALL workspaces and clusters           |
| Coarse table-level permissions only        | Fine-grained: row-level, column-level, file-level permissions        |
| No lineage tracking                        | Automated column-level lineage captured at runtime                   |
| No audit logs                              | Complete audit log of every data access event                        |
| No governance for ML models/dashboards     | Governs tables, files, ML models, dashboards, notebooks — everything |
| No cross-cloud sharing                     | Delta Sharing for secure cross-platform/cross-cloud data sharing     |

### 6.2 Three-Level Namespace ★ Critical Concept ★

```
METASTORE (one per region/account)

■■■ CATALOG (e.g., "production", "dev", "marketing")

■■■ SCHEMA / DATABASE (e.g., "sales", "analytics")

■■■ TABLE / VIEW / VOLUME / FUNCTION / MODEL

-- Query syntax:

SELECT * FROM catalog1.database1.table1;

-- Legacy Hive tables still accessible via:

SELECT * FROM hive_metastore.database2.table2;
```

| Level | Object            | Purpose                                                            |
|-------|-------------------|--------------------------------------------------------------------|
| 1     | Catalog           | Top-level isolation — one per business unit or SDLC env (dev/prod) |
| 2     | Schema (Database) | Groups related tables — one per project or team                    |
| 3     | Table             | Actual data — managed or external                                  |
| 3     | View              | Saved SELECT query over tables                                     |
| 3     | Volume            | Non-tabular data storage (files, images, PDFs, JSON blobs)         |
| 3     | Function (Model)  | UDFs and registered ML models (MLflow packaged)                    |

## 6.3 Managed vs External Tables ★ Classic Exam Question ★

| Feature             | Managed Tables                                                        | External Tables                                            |
|---------------------|-----------------------------------------------------------------------|------------------------------------------------------------|
| Who owns data?      | Unity Catalog manages BOTH metadata AND physical files                | You own the files; UC only manages the metadata pointer    |
| Storage location    | UC-managed storage (dedicated S3/ADLS bucket per metastore)           | YOUR S3/ADLS path — you specify LOCATION when creating     |
| DROP TABLE behavior | <b>Deletes BOTH metadata and physical data files (within 30 days)</b> | <b>Deletes ONLY metadata. Your data files remain SAFE.</b> |
| Use case            | New data projects, easiest option                                     | Existing data lakes, shared data with other tools          |

### ■ EXAM TRAP

DROP TABLE on Managed = data GONE. DROP TABLE on External = only pointer deleted, data SAFE. This is the #1 exam trap in Unity Catalog.

## 6.4 Unity Catalog Volumes

Volumes are UC objects for **non-tabular data governance**. They complement tables (structured data) by managing files of any format.

```
-- Access path:
/Volumes/////

-- Also works with Apache Spark:
dbfs:/Volumes/////

-- Create managed volume:
CREATE VOLUME catalog.schema.my_volume;

-- Create external volume:
CREATE EXTERNAL VOLUME catalog.schema.ext_vol
LOCATION 'abfss://container@account.dfs.core.windows.net/path/';

-- Drop volume:
DROP VOLUME IF EXISTS my_volume;
```

## 6.5 Automated Lineage

| Benefit                 | Description                                            |
|-------------------------|--------------------------------------------------------|
| End-to-end transparency | Shows how data flows from raw sources to BI dashboards |
| Improved data quality   | Trace errors or quality issues back to their origin    |

|                         |                                                                 |
|-------------------------|-----------------------------------------------------------------|
| Impact analysis         | Know what downstream reports/models break if you change a table |
| Governance & compliance | Permissions enforced; users without access see masked lineage   |

#### ■ KEY CONCEPT

- Lineage is generated by the **Databricks runtime (UC lineage engine)**, NOT by Delta Lake itself
- Captured for ALL languages: SQL, Python, Scala, R
- Column-level lineage: tracks exactly which source columns contribute to each output column
- Streaming lineage requires Delta Lake tables (Runtime 11.3 LTS+)

# CHAPTER 7

## Delta Sharing

### 7.1 What is Delta Sharing?

Delta Sharing is an **open protocol** (not proprietary Databricks) for securely sharing **live data** across organizations and platforms **without copying the data**.

#### ■ KEY CONCEPT

- Provider shares data → Recipient queries it in real time
- **No data replication** — recipient reads directly from provider's storage via pre-signed URLs
- Works across clouds, companies, and tools (Power BI, Tableau, Spark, Pandas, Java, etc.)
- Sharing happens on Delta part files: supports full tables, partitions, and delta versions

### 7.2 Two Sharing Modes

| Mode                           | When to Use                                          | Authentication                                                                                 | How Recipient Accesses Data                                   |
|--------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Databricks-to-Databricks (D2D) | Both parties use Databricks with Unity Catalog       | Sharing Identifier (unique metastore ID) — no tokens/files needed                              | Data appears directly in recipient's Catalog Explorer         |
| Open Sharing                   | Recipient uses ANY platform (Power BI, Python, etc.) | Provider creates Recipient object + activation link → recipient downloads JSON credential file | Recipient uses open-source Delta Sharing client to query data |

#### ■ EXAM TRAP

Even with D2D sharing: recipient workspace can only READ shared data — they CANNOT modify it.

### 7.3 How Delta Sharing Works (Under the Hood)

```
Step 1: Client authenticates to the Delta Sharing Server
Step 2: Client requests a table (with optional filters)
Step 3: Server checks access permissions (via Unity Catalog ACLs)
Step 4: Server generates and returns pre-signed short-lived URLs → Parquet files
Step 5: Client reads Parquet files DIRECTLY from object storage using those URLs

# The data NEVER passes through the sharing server itself!
# Server only hands out temporary access URLs.
```

# This is why Delta Sharing is scalable and eliminates data replication.

## 7.4 D2D Sharing Scenarios

| Scenario                  | Details                                                                    |
|---------------------------|----------------------------------------------------------------------------|
| Different cloud regions   | Same company, AWS us-east-1 → AWS eu-west-1                                |
| Different companies       | Company A (provider) → Company B (recipient), both on Databricks           |
| Different cloud providers | AWS provider → Azure recipient (cross-cloud!)                              |
| Different technologies    | Databricks provider → Power BI / Python / Tableau recipient (Open Sharing) |

# CHAPTER 8

*Quick Reference & Memory Tricks*

## 8.1 DataFrame Operations Cheatsheet

| Operation         | Code Pattern                                                   | Key Notes                                                       |
|-------------------|----------------------------------------------------------------|-----------------------------------------------------------------|
| Filter rows       | <code>df.filter(col("price") &gt;= 300)</code>                 | Use <code>col()</code> from <code>pyspark.sql.functions</code>  |
| Select columns    | <code>.select("name", "cat")</code>                            | Returns only specified columns                                  |
| Order results     | <code>.orderBy(col("price").desc())</code>                     | Default is ASC; use <code>.desc()</code> for descending         |
| Add/transform col | <code>.withColumn("new", when(cond, val).otherwise(v2))</code> | <code>when/otherwise</code> = if-else for columns               |
| Group + aggregate | <code>.groupBy("cat").agg(sum("qty").alias("total"))</code>    | Import <code>sum</code> from <code>pyspark.sql.functions</code> |
| Inner join        | <code>df1.join(df2, "id", "inner")</code>                      | Drops non-matching rows from both sides                         |
| Full outer join   | <code>df1.join(df2, "id", "full")</code>                       | Keeps all rows; NULLs where no match                            |
| Left anti join    | <code>df1.join(df2, "id", "left_anti")</code>                  | Only left rows with NO match in right                           |
| Filter nulls      | <code>.filter(col("x").isNull())</code>                        | Use <code>isNull()</code> not <code>== None</code>              |

## 8.2 Delta Lake Operations Cheatsheet

| Feature          | Syntax / Notes                                                                                                                   |
|------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Read Delta       | <code>spark.read.format("delta").load("/path/")</code>                                                                           |
| Write Delta      | <code>df.write.format("delta").mode("overwrite").save("/path/")</code>                                                           |
| Schema Evolution | <code>.option("mergeSchema", "true")</code> on write                                                                             |
| MERGE (Upsert)   | <code>deltaTable.alias("t").merge(src.alias("s"), "t.id=s.id").whenMatchedUpdate(...).whenNotMatchedInsert(...).execute()</code> |
| VACUUM           | <code>VACUUM table RETAIN N HOURS</code> — deletes old unreferenced Parquet files                                                |
| Time Travel      | <code>spark.read.format("delta").option("versionAsOf", 5).load("/path/")</code>                                                  |
| Z-Order          | <code>OPTIMIZE table ZORDER BY (col1, col2)</code>                                                                               |
| History          | <code>DESCRIBE HISTORY delta.`/path/`</code>                                                                                     |
| Restore          | <code>RESTORE TABLE t TO VERSION AS OF 3</code>                                                                                  |

## 8.3 Unity Catalog Cheatsheet

| Concept            | Key Facts                                                             |
|--------------------|-----------------------------------------------------------------------|
| 3-Level Namespace  | catalog.schema.table — always reference tables this way in UC         |
| Managed Table      | UC owns data + metadata; DROP TABLE = permanent delete within 30 days |
| External Table     | You own data; DROP TABLE = only metadata deleted; files STAY          |
| Volume             | For non-tabular data; path /Volumes/catalog/schema/vol/file           |
| Lineage            | Auto-captured by Databricks runtime; column-level; all languages      |
| Delta Sharing      | Open protocol; no data copy; D2D or Open Sharing modes                |
| Storage Credential | IAM Role (AWS) or Managed Identity (Azure) for cloud storage access   |
| External Location  | Specific cloud path + Storage Credential → UC can access that path    |

## 8.4 ★ TOP MEMORY TRICKS ★

### ■ MEMORIZE THESE — Most Likely Exam Points

**product\_id 999 has NO match** — the "orphan row" trick in EVERY join question

**Inner join** REMOVES non-matching rows (999 gone). **Full outer** KEEPS them (999 appears as NULL).

**Left Anti** = "rows from left that were NOT invited to the party on the right"

**Z-Order** clusters data so min-max stats per file become TIGHT → better data skipping

**VACUUM 168 hours** = 7 days retention. Files deleted 10 days ago = PERMANENTLY GONE

**MoR**: writes small delta files; original Parquet files INTACT (immutable). Merge happens at READ time.

**Schema Enforcement** = reject bad writes. **Schema Evolution** = accept new columns (mergeSchema=true).

**UC Managed Table**: DROP = data deleted. **External Table**: DROP = only pointer deleted, data SAFE.

**ArrayType** = list of same-type values. **StructType** = nested record with named fields.

**Delta Sharing** = open protocol, no data copy, pre-signed URLs, works cross-cloud.